

Contract Analyzer AI App

Full-Stack Implementation & Deployment Report

TECHNOLOGY STACK:

Frontend: React, Vite, Tailwind CSS, Axios, Lucide Icons

Backend: FastAPI, Uvicorn, Google Antigravity (ADK) SDK, PyPDF

AI Engine: Gemini API (gemini-2.5-flash)

Infrastructure & Deployment: Docker, Google Cloud Build, Google Cloud Run

Generated on: August 21, 2026

Designed and built by Antigravity Agentic AI

1. Executive Summary

We have successfully built, verified, and deployed the Contract Analyzer, a modern, full-stack AI web application that automatically parses, extracts, and summarizes legal contracts, and provides a conversational interface to ask questions about the contract.

The project leverages Google's official Antigravity (ADK) Agent framework to drive the interactive chatbot on the frontend, and uses the standard Google GenAI SDK to perform highly accurate, structured data extraction representing parties, key dates, obligations, and risk factors in the legal documents.

Initially designed as separate services, the final deployment was optimized into a unified single container hosted on Google Cloud Run. The React UI is served directly from the FastAPI root path, eliminating CORS complexities, minimizing cloud compute costs, and providing a single, seamless live application URL.

2. Application Architecture

The application architecture consists of a compiled single-page React frontend mounted directly inside a FastAPI backend server. The backend runs inside a lightweight Docker container on Google Cloud Run.

Unified Directory Structure

```
contract-analyzer/
|-- backend/
|   |-- agents/
|   |   |-- contract_agent/
|   |       |-- __init__.py
|   |       |-- agent.py           # Antigravity ADK Agent configuration
|   |-- static/                   # Production build of the React app
|   |   |-- assets/               # CSS & JS assets
|   |   |-- index.html
|   |-- Dockerfile
|   |-- main.py                   # FastAPI backend server with routes
|   |-- requirements.txt
|   |-- .env.example
|-- frontend/
|   |-- src/
|   |   |-- App.jsx               # React main dashboard UI
|   |   |-- index.css             # Tailwind base & custom scrollbars
|   |   |-- main.jsx
|   |-- index.html
|   |-- package.json              # Frontend npm dependencies
|   |-- vite.config.js
```

3. Step-by-Step Implementation Details

Phase 1: Backend & ADK Agent Setup

We configured the `contract_agent` using Google ADK LLM Agent framework. The agent is configured with the `gemini-2.5-flash` model and legal-expert system instructions.

In `main.py`, we resolved a startup crash issue: Google GenAI Client (`genai.Client()`) crashes on startup if `GEMINI_API_KEY` environment variables are missing during container initialization. To solve this, we implemented lazy initialization of the client at request time, ensuring the container starts successfully and passes the Cloud Run health checks.

Phase 2: Structured Contract Analysis

FastAPI provides a `POST /api/analyze` endpoint that handles file uploads (.pdf and .txt). Text is extracted using the `pypdf` library. We defined a strict Pydantic output schema (`ContractAnalysisResult`) to guarantee the integrity of the extracted metadata. Gemini 2.5 is then called with this schema to output structured JSON containing parties, effective/expiration dates, key clauses, obligations, and risk assessment grades.

Phase 3: React Frontend Dashboard

The React UI provides a clean side-by-side layout using Tailwind CSS. On the left panel, the uploaded contract's extracted metadata is displayed in cards (such as color-coded Risk badges based on low/medium/high severity). On the right panel, users can chat with the ADK agent.

The frontend is configured with environment-aware host detection, defaulting to relative paths if hosted on the same domain (production) or falling back to the cloud URL if running local Vite dev server.

4. Deployment & Cloud Hosting

We compiled the React app into static files (index.html, JS/CSS assets) and mounted them inside FastAPI:

```
static_dir = os.path.join(backend_dir, 'static')
if os.path.exists(static_dir):
    @app.get('/')
    async def read_index():
        return FileResponse(os.path.join(static_dir, 'index.html'))
    app.mount('/assets', StaticFiles(directory=os.path.join(static_dir, 'assets')))
```

GCP Artifact Registry and Cloud Run

The container image was successfully compiled using Google Cloud Build and pushed to the standard Artifact Registry repository (adk-repo) at region us-central1:

us-central1-docker.pkg.dev/project-83b9dd42-537f-47c6-a28/adk-repo/contract-analyzer-backend:latest

The container was deployed to Cloud Run and is live at:

<https://contract-analyzer-backend-450158463415.us-central1.run.app>

5. Verification & Live Status

We verified the deployment is live by calling the endpoints:

1. GET /health: Returns {"status":"ok"} confirming container status.
2. GET /docs: Exposes Swagger interactive docs for the backend APIs.
3. GET /: Serves the React single-page app index file, loading compiled assets successfully.

The application is fully operational and ready to parse and chat about legal documents in real-time.